

Peripheral Interrupts

Peripheral Interrupts

- 1. Peripheral Interrupt Overview
- 2. Core Concepts
 - 2.1 External Interrupt Principle
 - 2.2 ISR Hard Constraints
 - 2.3 `volatile` Keyword
 - 2.4 Debounce Strategy
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Macro Definitions and Global Variables
 - 5.2 `setup()` — Initialization and Interrupt Registration
 - 5.3 `loop()` — Debounce and LED Toggle
 - 5.4 `keyISR()` — Interrupt Service Routine
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Normal Operation
 - 6.3 Boundary Testing
- 7. Common Issues

1. Peripheral Interrupt Overview

External Interrupt (External Interrupt / EXTI) is a hardware mechanism by which the MCU responds to external asynchronous events. When a GPIO pin detects a specified edge (rising edge/falling edge/either edge), the hardware automatically pauses the current program execution, jumps to a preset **Interrupt Service Routine (ISR)** to handle the event, and returns to the original program to continue execution after handling is complete.

This experiment uses the BOOT button (GPIO0) to trigger an external interrupt. The ISR only records a timestamp, while debouncing and LED toggling are handled in `loop()`. This "ISR marks flag + main loop does the work" pattern is the universal paradigm for all interrupt applications.

Typical Application Scenarios:

Application Type	Example
Edge trigger	Emergency stop button, speed sensor, PIR motion detection
Level trigger	Power loss detection, overcurrent protection signal
ISR + main loop	Touch IC status pin, encoder Z-phase home signal

2. Core Concepts

2.1 External Interrupt Principle

`attachInterrupt(digitalPinToInterrupt(pin), ISR, mode)` maps GPIO edge events to the CPU interrupt vector table. When the pin level changes by the specified amount, the CPU pauses the current task, jumps to the ISR function for execution, and returns to the original task to continue running after execution is complete.

mode parameter	Trigger condition
<code>CHANGE</code>	Triggers on both rising and falling edges
<code>RISING</code>	Rising edge only (LOW to HIGH)
<code>FALLING</code>	Falling edge only (HIGH to LOW)

This experiment uses `FALLING` edge triggering because it needs to capture the press event to determine bounce.

2.2 ISR Hard Constraints

The following operations are **NOT** allowed in ISR, otherwise they will cause deadlock, watchdog reset, or data corruption:

Prohibited operation	Reason
<code>Serial.print()</code>	Internally relies on UART interrupt, will cause deadlock in ISR
<code>delay()</code> / long operations	Higher priority interrupts are masked during ISR, delaying all timing/communication
<code>malloc()</code> / <code>free()</code>	May trigger kernel scheduling, not allowed in ISR context
Modifying non- <code>volatile</code> global variables	Compiler may cache in registers, <code>loop()</code> cannot see updates

Correct approach: ISR only does the simplest operations (sets flag, records timestamp), leaving heavy work to `loop()`.

2.3 `volatile` Keyword

```
volatile unsigned long lastEdgeTime = 0; // written by ISR, read by loop()
```

The compiler optimizer caches "seemingly unchanged" variables in CPU registers. `volatile` forces each reload from RAM, ensuring that values written by ISR can be immediately seen by `loop()`. The typical symptom of not adding `volatile` is: ISR clearly updates the variable, but `loop()` always reads the old value.

2.4 Debounce Strategy

This experiment uses a separated strategy of **ISR records timestamp + loop() debounces**:

- In ISR, only `millis()` records the latest edge time ($< 1\mu s$)
- In `loop()`, judge "more than 30ms since last edge" as stable, then read the level

There are two benefits to placing debouncing in `loop()` instead of ISR: ISR remains very short; debounce logic can be debugged with `Serial.print`.

3. Hardware Dependencies

Pin	Peripheral	Configuration	Description
GPIO44	LED	OUTPUT	Common anode connection, low level lights up
GPIO0	BOOT button	INPUT_PULLUP	Press = low level, falling edge interrupt

4. Project Architecture and Flow

```
setup()
├─ pinMode(LED, OUTPUT)           // LED push-pull output
├─ pinMode(KEY, INPUT_PULLUP)     // Button pull-up input
├─ digitalWrite(LED, OFF)         // Initial off
└─ attachInterrupt(KEY, keyISR, FALLING) // Register falling edge interrupt

loop()                             // Main loop
├─ if (now - lastEdgeTime >= 30ms) // Debounce: more than 30ms since last edge?
│   ├─ reading = digitalRead(KEY) // Read stable level
│   └─ if (reading != lastStableState) // Different from last stable value?
│       ├─ Update lastStableState
│       └─ if (reading == LOW) // Falling edge → valid press
│           └─ Toggle LED

keyISR()                           // Interrupt Service Routine
└─ lastEdgeTime = millis()         // Only record timestamp, nothing else
```

5. Source Code Details

Full source code: `Source Code` → `Arduino` → `1.Basic Peripherals` → `3.NVIC_KEY` → `3.NVIC_KEY.ino`

5.1 Macro Definitions and Global Variables

First define pin numbers and debounce time. Two global variables use `volatile` and regular type to distinguish ISR access permissions.

```

#define LED_PIN      44
#define KEY_PIN      0
#define DEBOUNCE_MS 30

volatile unsigned long lastEdgeTime = 0; // written by ISR, read by loop(), must be
volatile
bool ledState = false;                  // Only accessed by loop(), no volatile needed
bool lastStableState = HIGH;            // Last stable level after debounce

```

`lastStableState` initial value `HIGH`: In pull-up state, unpressed is HIGH, consistent with physical state.

5.2 `setup()` — Initialization and Interrupt Registration

Configure GPIO and register interrupt. `attachInterrupt` internally completes pin to interrupt number mapping and vector table writing.

```

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(KEY_PIN, INPUT_PULLUP);
  digitalWrite(LED_PIN, ledState);

  attachInterrupt(digitalPinToInterrupt(KEY_PIN), keyISR, FALLING);
}

```

`digitalPinToInterrupt()` converts Arduino pin number to internal interrupt number. Not all GPIO support interrupts — ESP32-S3 all GPIO are usable, but AVR boards like Uno have only a few.

5.3 `loop()` — Debounce and LED Toggle

Debounce judgment is done in `loop()`, not blocking ISR. Only when more than 30ms has passed since last edge is the signal considered stable.

```

void loop() {
  unsigned long now = millis();

  if (now - lastEdgeTime >= DEBOUNCE_MS) { // More than 30ms since last edge?
    bool reading = digitalRead(KEY_PIN);    // Read stable level

    if (reading != lastStableState) {       // Different from last confirmed value?
      lastStableState = reading;
      if (reading == LOW) {                 // Falling edge → valid press
        ledState = !ledState;
        digitalWrite(LED_PIN, ledState);
      }
    }
  }
}

```

The debounce logic is essentially the same as the `delay(20)` in the button experiment, but here it does not block — `loop()` continues running, just skips readings during the bounce period. `lastStableState` prevents secondary toggles caused by release bounce.

5.4 `keyISR()` — Interrupt Service Routine

ISR only does one thing: records the timestamp. This is the best practice for ISR — very short, non-blocking, no peripheral operations.

```
void keyISR() {  
  lastEdgeTime = millis(); // only record timestamp, fastest operation  
}
```

Is `millis()` safe in ISR? Safe — it only reads a hardware counter register, does not depend on interrupts, and does not block.

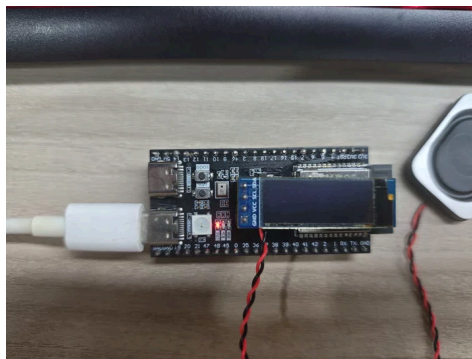
6. Operation Flow

6.1 Build and Flash

Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

6.2 Normal Operation

- After power-on, LED is initially **off** (`ledState = false`)
- Each press of BOOT button → LED toggles once



6.3 Boundary Testing

Test Item	Operation	Expected Result
Short press	Quick tap BOOT button	LED toggles once
Long press	Hold BOOT button	LED toggles once (debounce judges only one falling edge)
Fast continuous press	Continuous quick taps	Each valid press toggles once

7. Common Issues

Phenomenon	Cause	Solution
Adding <code>Serial.print</code> in ISR causes hang	Serial port forbidden in ISR	Move print to <code>loop()</code>
LED does not toggle	<code>volatile</code> missing	Check if <code>lastEdgeTime</code> is declared as <code>volatile</code>
One press toggles multiple times	Debounce time insufficient	Increase <code>DEBOUNCE_MS</code> to 50ms
Interrupt does not trigger	GPIO0 mode error	Confirm <code>pinMode(KEY_PIN, INPUT_PULLUP)</code>